

LOCAL REVIEWED REBUILD

The Best New AI Opportunity Nobody Is Talking About

Agent-native apps for Codex, Cursor, and bring-your-own-agent workflows

context

tools

shared surface

memory loop

Thesis: build apps that a user's own agent can enter, operate, and improve.

BYO agent replaces app-owned assistant

The opportunity is an inversion

BLUF: the product surface moves inside the user's agent workflow.

Agent-native apps make the product a shared surface for the user's agent.

1. The old model is bounded

An app-owned assistant can only use the context and actions exposed inside that product.

2. The new model is portable

The user's agent brings files, project memory, browser context, tools, and judgment into many apps.

3. The wedge is the surface

The product opportunity is a reviewable workspace: canvas, document, board, sheet, or operational console.

Strategic reading: do not build another isolated chatbot. Build a surface that can be operated through MCP, CLI, API, browser automation, and explicit human approval.

The moat is not the chat box. The moat is context, tool access, and visible collaboration.

Why this is happening now

The enabling pieces have converged into a practical build pattern.

Narrative

Local coding agents, browser-hosted work surfaces, and MCP-style tool access now let software operate as part of the user's agent workflow instead of as an isolated destination.

- Codex runs locally and can inspect or change project files.
- MCP standardizes connections to data, tools, and workflows.
- Web apps can provide a visible shared surface.

Local agents

1

Codex/Cursor inspect files, run commands, and coordinate work.

Tool standard

2

MCP exposes external systems as model-invoked tools.

Web surfaces

3

Browser-hosted apps can be visible to human and agent.

Permission loops

4

Sensitive actions can require explicit approval.

Convergence creates the wedge

The opportunity exists because context, tools, and surfaces can now meet in one workflow.

SECTION 01

The software model is changing

From app-owned assistants to user-owned agents.

1. Old boundary

Each app owns its small assistant.

2. New workspace

The user's agent carries context across tools.

3. Product shift

The surface becomes the leverage point.

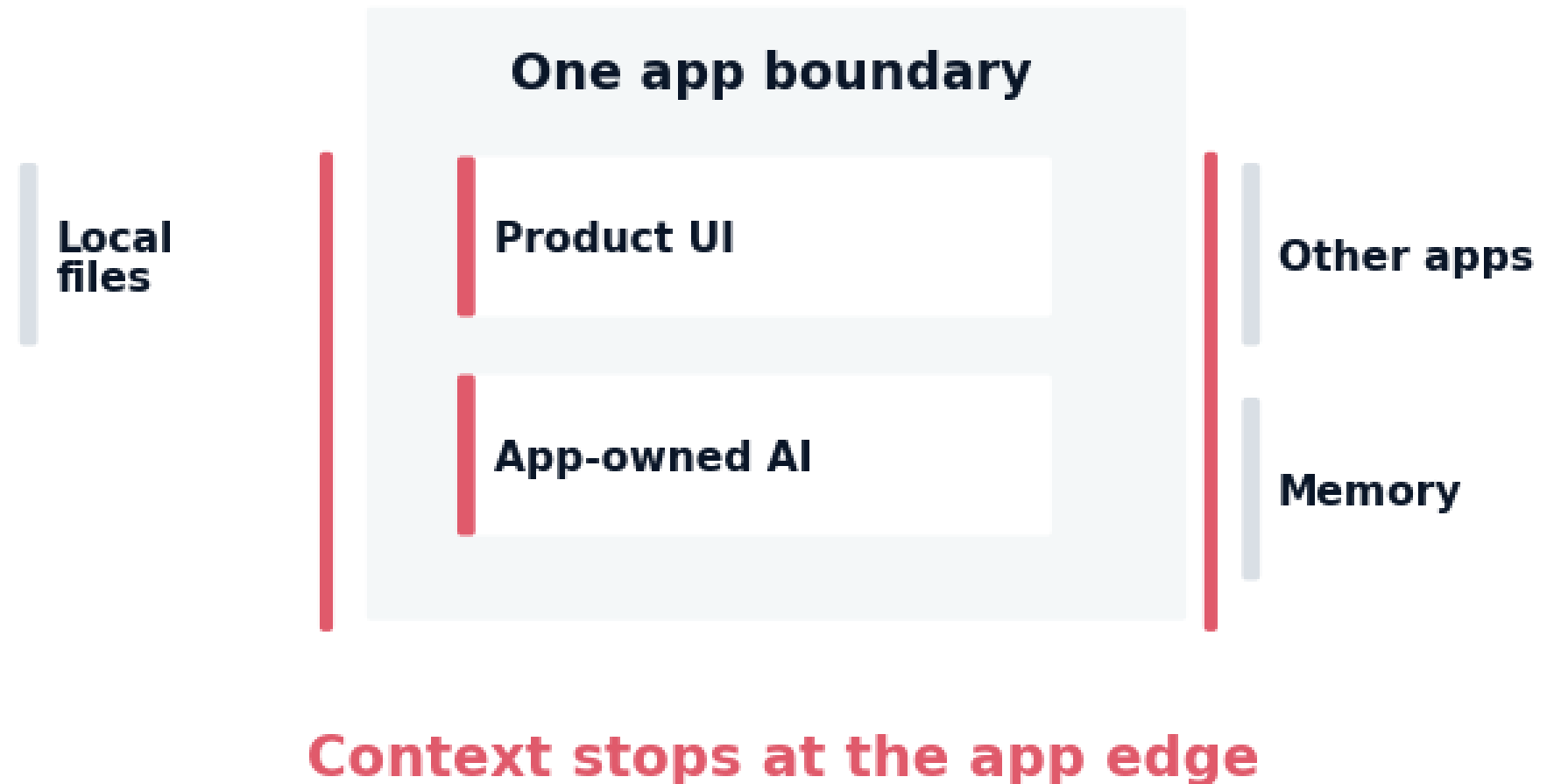
The old model is app with agent

AI is bolted onto one product boundary.

Narrative

Most early AI apps added an assistant inside the existing product. That helps with app-local tasks, but it cannot naturally use the user's broader project context, local files, or other work systems.

- The assistant is scoped to one application's data model.
- Connectors help, but the app still controls the agent boundary.
- The user must repeat context across many isolated assistants.



App-owned AI improves one silo; it does not become the user's operating layer.

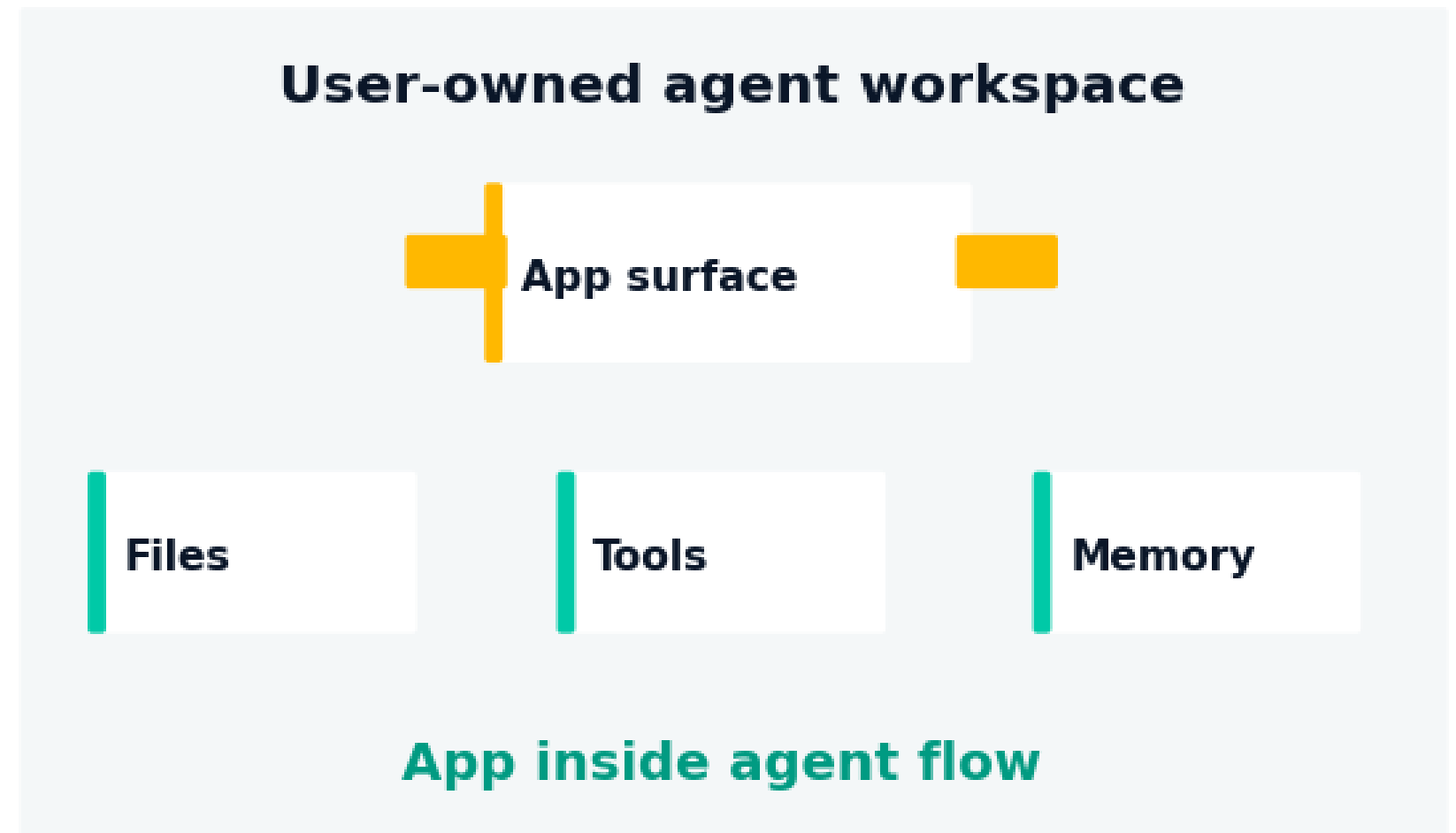
The new model is agent with app

The user's own agent becomes the outer workflow.

Narrative

Agent-native apps invert the control model. The app is a surface inside the user's agent workspace, while the agent brings local context, tool access, memory, and approvals around it.

- The user's agent can reference files and project history.
- The app exposes operations through MCP, CLI, API, or browser actions.
- The human sees the work and corrects the loop early.



The product is no longer only a destination; it becomes a surface agents can operate.

Agent-native has a precise definition

It is not the same thing as having a chatbot.

Narrative

An agent-native app is designed so an external user-owned agent can understand the state, take useful actions, and leave behind improved context. The app does not need to own the whole intelligence layer.

- Context is readable by the agent.
- Actions are invocable through explicit tool contracts.
- State changes are visible and reviewable.



Agent-native means agent-operable, human-visible, and memory-aware.

The user buys completed work

The job is shared between human, agent, and app.

Narrative

The user is not buying another chat interface. They are buying an outcome: a design explored, a document updated, a sheet reconciled, or a workflow completed with enough visibility to trust the result.

- The human supplies judgment and approval.
- The agent supplies context handling and execution.
- The app supplies the durable work surface.



Outcome: completed work,
~~not generated chat~~

Frame the product around the completed job, not the presence of AI.

SECTION 02

Architecture makes it real

A strong agent-native app has four building blocks.

1. Context

What the agent knows.

2. Tools

What the agent can do.

3. Surface

Where work stays visible.

Four building blocks matter most

The architecture is simple, but the details matter.

Narrative

Agent-native architecture combines context, connectivity, collaboration, and compounding memory. Weak products usually miss one of these blocks and collapse back into a chatbot or brittle automation.

- Context prevents repeated prompting.
- Connectivity turns suggestions into actions.
- Compounding memory improves future runs.

Context

What the agent needs to know before acting.

Connectivity

How the agent reads, writes, and invokes.

Collaboration

Where human and agent see the same object.

Compounding

How approved work becomes future memory.

If one block is missing, the agent cannot reliably finish real work.

Reference architecture is a loop

The product should be designed as an operating cycle.

Narrative

The agent reads context, plans work, invokes tools, edits the shared surface, then writes approved outcomes back to memory. That loop is the core system, regardless of the specific domain.

- The agent host is not the whole product.
- The tool layer should be explicit and testable.
- The shared surface makes progress inspectable.

1. Context sources

2. Agent host

3. Tool layer

4. Shared surface

5. Write-back

Design the loop first; feature lists come after.

Context is the first product layer

Files and project truth are not optional.

Narrative

The biggest advantage of bring-your-own-agent software is that the agent can start with the user's actual project materials: local markdown, code, screenshots, brand rules, customer notes, and prior decisions.

- Use predictable folders and filenames.
- Keep durable project context in markdown or structured data.
- Write decisions back after approval.



Context is the product's operating memory

Context turns the app from a blank surface into a project-aware workspace.

Tools convert intent into work

MCP, CLI, API, and browser actions are the execution layer.

Narrative

Tool access is where agent-native products become useful. The agent needs small, composable primitives that map to real app operations, with safe permissions and clear results.

- Expose read and write capabilities intentionally.
- Prefer complete CRUD for important entities.
- Use confirmation for risky operations.



Do not hide the app behind one workflow-shaped tool; expose useful primitives.

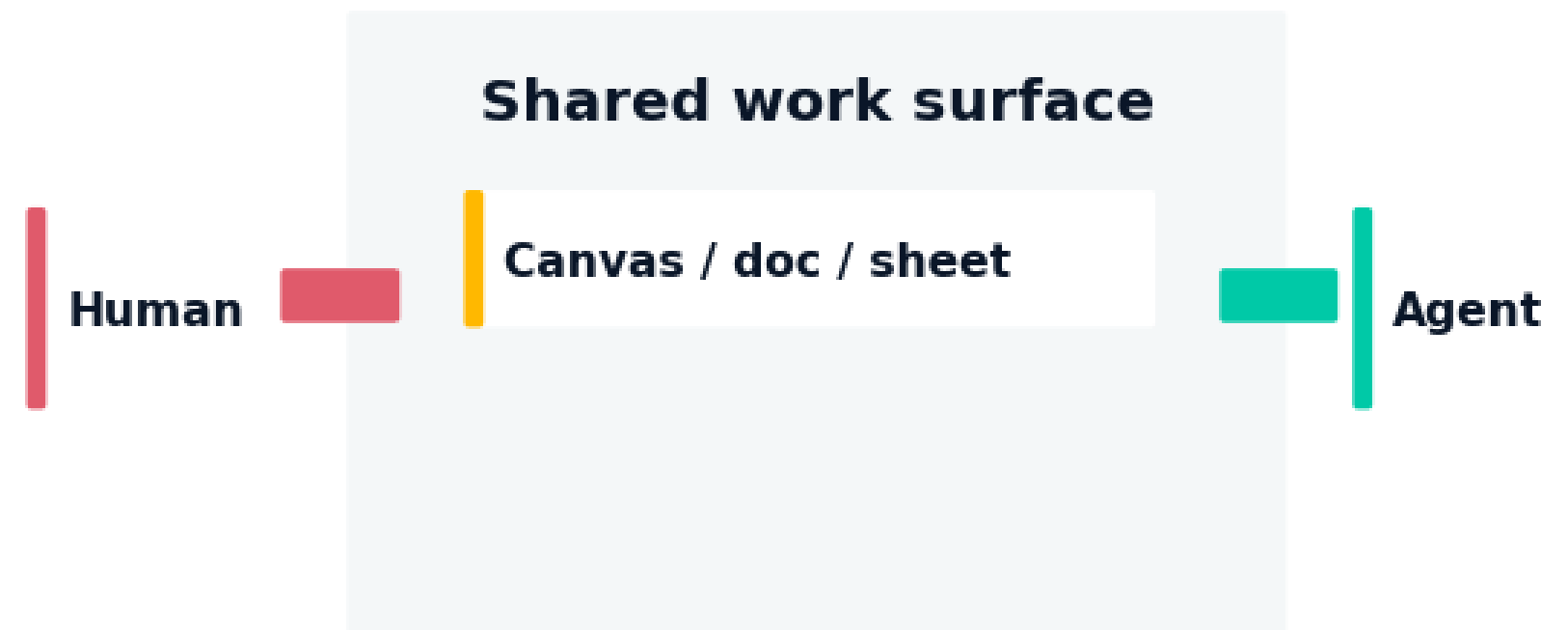
The surface must be shared

Human and agent need to see the same object.

Narrative

A shared surface can be a design canvas, Notion page, spreadsheet, dashboard, board, or operational console. What matters is that both human and agent work on the same visible state.

- Visual state makes mistakes easy to catch.
- The user can intervene without resetting the workflow.
- The agent can use the surface as feedback.



Visible state makes correction cheap

Visible collaboration is the trust mechanism.

Memory makes quality compound

Approved work should improve the next run.

Narrative

Agent-native software gets stronger when completed work becomes reusable context: brand choices, page structures, prompt corrections, customer rules, and workflow decisions should feed the next task.

- Capture decisions, not every token.
- Separate durable memory from raw logs.
- Make write-back explicit and reviewable.



Approved work becomes reusable context

The product gets better when the project remembers what worked.

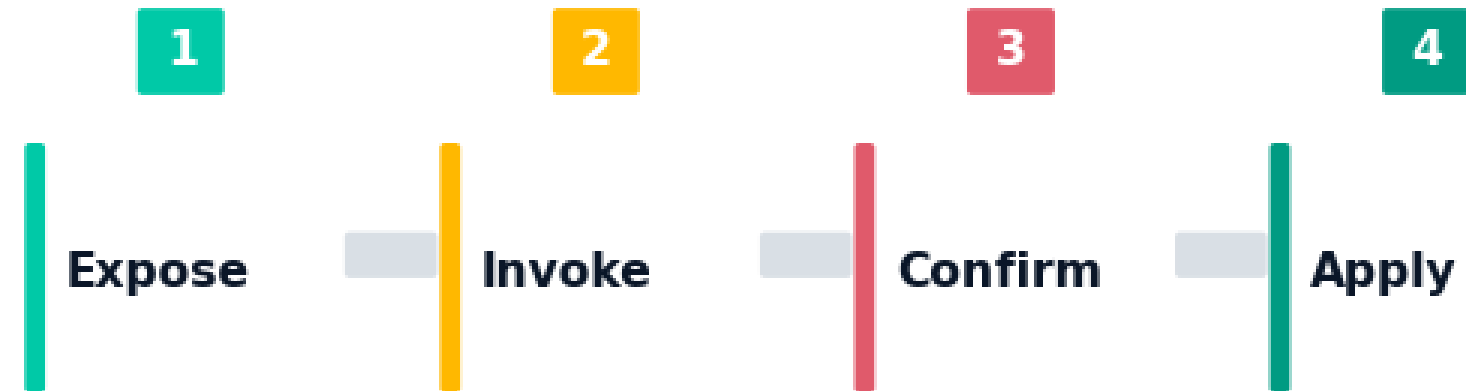
Trust needs visible approval gates

The MCP spec's human-in-loop guidance is product design advice.

Narrative

External tool use should be legible: users need to know which tools are exposed, when tools are invoked, and when a sensitive action requires confirmation. That is not bureaucracy; it is the interface for trust.

- Show available tools and permissions.
- Make tool invocation visible in the UI.
- Require confirmation for irreversible actions.



Trust is designed into the interface

Trust is built through observable actions and reversible loops.

SECTION 03

Where the wedges are

The strongest opportunities look like familiar tools rebuilt for agents.

1. Design

Canvas work is visual and reviewable.

2. Knowledge

Docs already hold context.

3. Operations

Sheets and boards already drive action.

Design tools show the pattern

MagicPath-style workflows make the agent useful without owning all context.

Narrative

In the transcript's example, the agent reads local design and brand context, operates a design canvas in the browser, then can bring the resulting direction back to the codebase. The design tool is the surface, not the only brain.

- The app provides visual canvas state.
- The agent brings local files and project rules.
- The human reviews the design while it forms.



The agent brings the context

Design is a strong wedge because output is visible, iterative, and context-heavy.

Knowledge tools become live systems

Notion-style workspaces are more than repositories.

Narrative

When an agent can read and write a knowledge workspace with permission, documents become operational surfaces. The app can hold plans, customer notes, specs, and task boards while the user's agent coordinates updates.

- Notion MCP provides a read/write bridge based on permissions.
- Pages and databases become agent-operable entities.
- The human still owns review and workspace structure.



Docs become agent-operable state

Knowledge apps are attractive because the context and work surface already coexist.

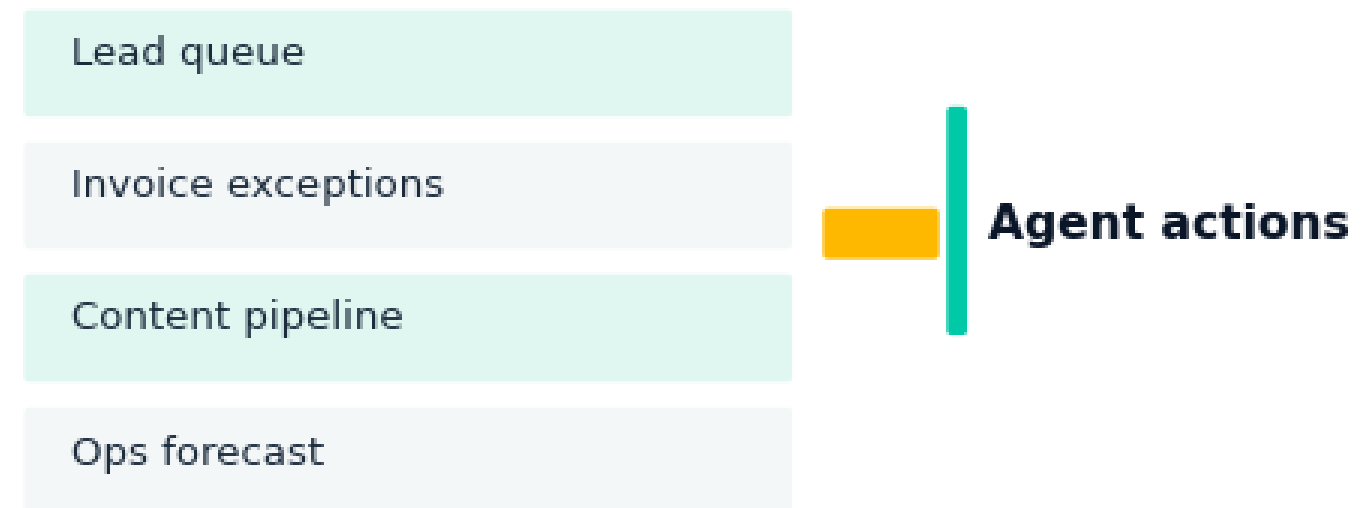
Sheets can become control planes

Rows are visible, structured, and easy to verify.

Narrative

Spreadsheets are often the operating layer for small businesses. Agent-native sheet tools can let an agent reconcile, explain, classify, forecast, or trigger follow-up while every row remains inspectable.

- Structured rows make state explicit.
- Humans can review before bulk changes.
- Actions can be scoped to small reversible batches.



Rows become action surfaces

Operational surfaces are good wedges when every action leaves a visible trace.

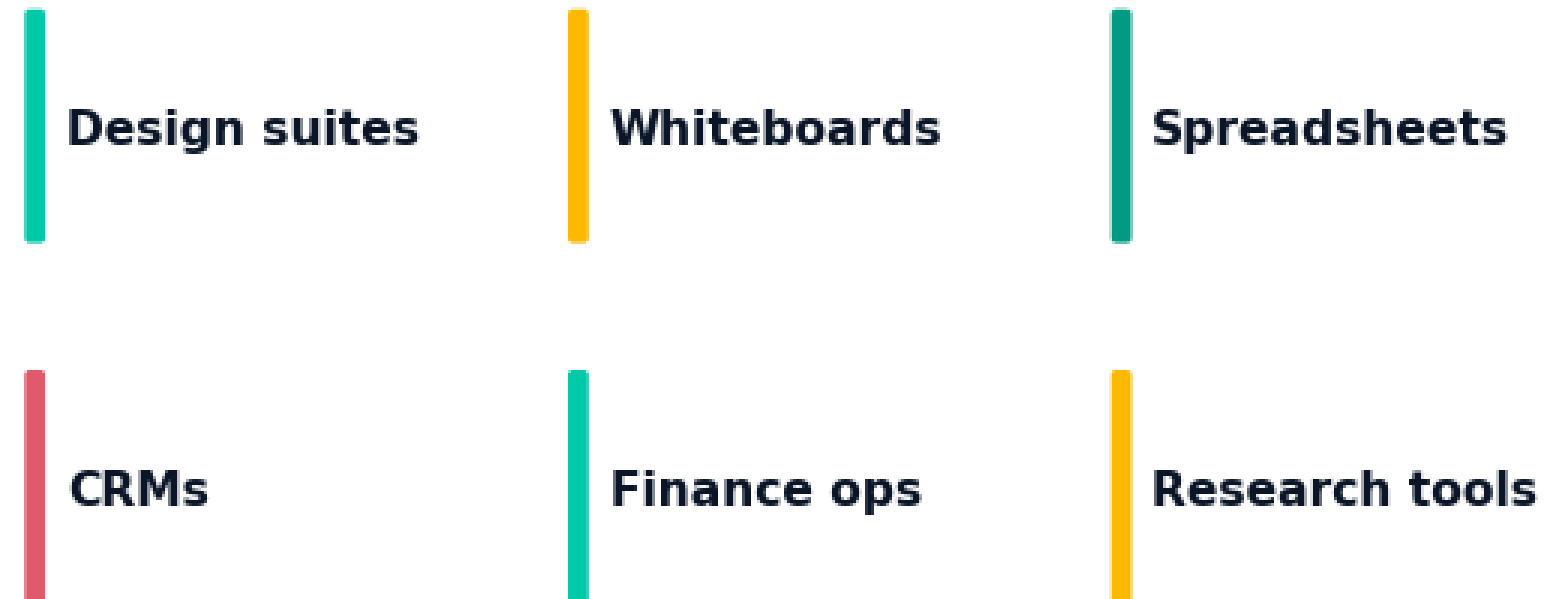
Legacy surfaces are exposed

The opportunity is not limited to new categories.

Narrative

Many incumbent tools still assume the human will manually move every object through the interface. Agent-native alternatives can start by targeting a narrow repeated workflow inside those surfaces.

- Prioritize high-context, repeated work.
- Avoid tasks where correctness is invisible.
- Start where existing tools force context switching.



Manual UI steps are the wedge

The best startup wedge may be a familiar surface redesigned for user-owned agents.

Pick repeatable visible work

The wedge should be easy to review before it is trusted.

Narrative

The strongest opportunities sit where the work happens often and the resulting state is visible. That lets the agent create leverage while the human can still understand and correct the output.

- High repetition creates habit and ROI.
- Visible state reduces trust friction.
- Domain context creates defensibility.



If users cannot see whether the agent is right, the wedge is too risky for v1.

Use a simple wedge rubric

Score candidates before writing code.

Narrative

A good first product should score well on repetition, visible state, context richness, actionability, and manageable risk. These are judgment scores, not invented market metrics.

- Favor workflows with repeated review cycles.
- Prefer surfaces where partial progress is useful.
- Deprioritize irreversible high-stakes automation.

	Repeat	Visible	Context	Action	Risk
Design review	H	H	H	H	M
Ops sheet	H	H	H	M	H
Legal drafting	H	H	M	H	H

Score, then test with tasks

A narrow but reviewable wedge beats a broad agent that users cannot trust.

SECTION 04

How to build it

Start with the job, then the surface, then the tool contract.

1. Job

Define the outcome.

2. Surface

Make work visible.

3. Contract

Expose safe actions.

Step one is the job

Define what human and agent achieve together.

Narrative

Start with a concrete job-to-be-done: create a page design, reconcile a list, update a campaign plan, draft a board, or prepare a report. Then specify what the human and agent each contribute.

- Name the completed artifact.
- Define what good looks like.
- List the decisions that require human judgment.



Vague job means vague tools

The app architecture should follow the job, not the AI demo.

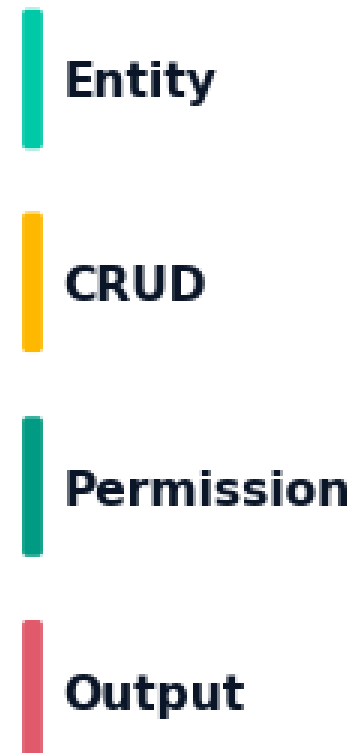
Step two is the tool contract

The agent needs complete, safe primitives.

Narrative

Plan the MCP, CLI, API, and browser operations from day one. For each important entity, define how the agent can read, create, update, delete, validate, and explain changes.

- Map tools to user vocabulary.
- Separate safe actions from approval-required actions.
- Return structured results the agent can reason about.



Tool contract = agent API

The tool contract is the hidden product surface for agents.

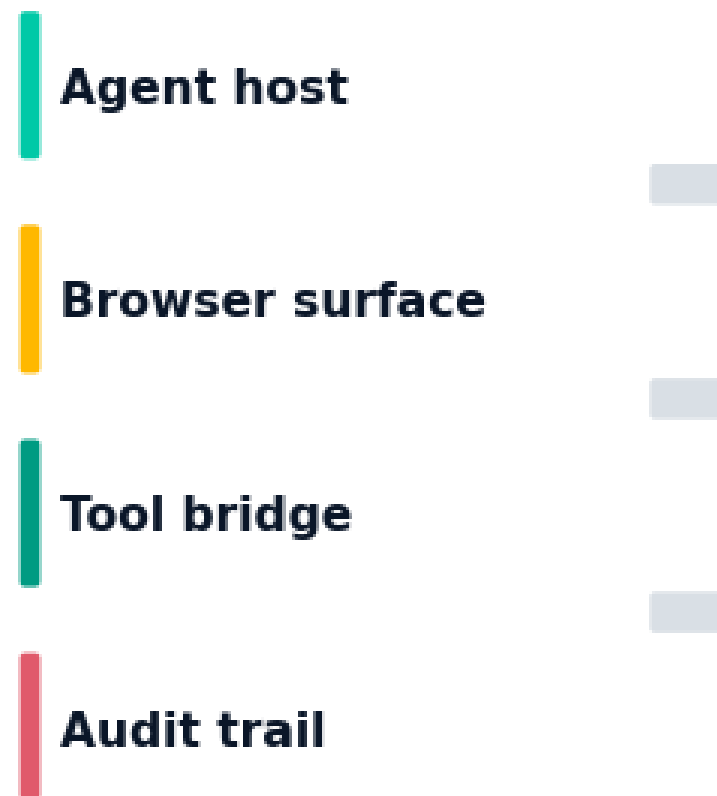
Step three is the harness

Test inside the actual agent workspace.

Narrative

Do not test only in your own app. Test inside Codex, Cursor, or the agent host your customer uses. The core question is whether the user's agent can complete the job with your surface and tools.

- Open the app where the agent can see it.
- Run task prompts against real project context.
- Capture logs, diffs, approvals, and write-back.



If it only works in your demo harness, it is not yet agent-native.

Avoid fake agent-native patterns

Some AI products look modern but keep the old architecture.

Narrative

The most common failure is using an agent as a router for hard-coded workflows. The product may talk like an agent, but it cannot handle new outcomes, share state, or compose tools in unexpected ways.

- Avoid one giant workflow tool.
- Avoid silent actions that users cannot inspect.
- Avoid UI actions with no matching agent capability.



Fake agentic = brittle automation

Agent-native products expose capability; they do not trap intelligence behind brittle flows.

Launch only after parity checks

The acceptance test is a real agent task.

Narrative

A launchable agent-native app should pass a simple test: can the user's agent accomplish a valuable outcome inside the app, using real context, while the human can see and approve the important changes?

- Every UI action needs an agent path.
- Every key entity needs complete CRUD.
- Every risky operation needs an approval gate.

UI parity

Full CRUD

Visible tool use

Approval gates

Shared state

Memory write-back

Launch when the agent completes a real job

Treat parity, visibility, and approval as launch requirements.

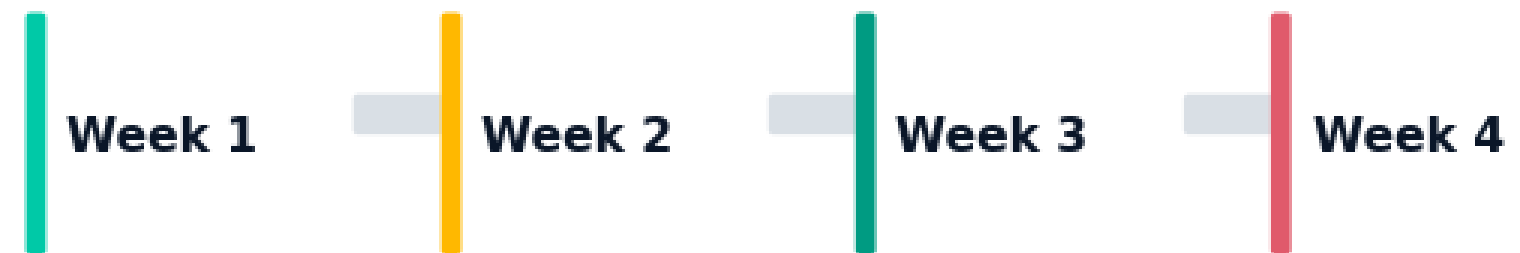
Recommendation: build the narrow loop

One wedge, one surface, one agent workflow.

Narrative

The right next move is a narrow v1: choose one repeated job, design the shared surface, expose the smallest complete tool contract, and test it inside the agent host your buyer already uses.

- Do not start with a general-purpose agent.
- Do not over-invest in decorative AI UX.
- Do build the loop users can trust and repeat.



One job, one surface, one loop

Win the first workflow, then let context and tool coverage expand.

Sources and validation notes

Claims are grounded in the transcript plus official product/protocol docs.

Narrative

This deck uses the supplied video transcript for the core thesis and current official documentation for product/protocol claims. No market-size numbers or adoption statistics were invented.

- OpenAI Codex docs and GitHub repo.
- Model Context Protocol docs and tools specification.
- Notion MCP and Cursor documentation.



OpenAI Codex



MCP docs/spec



Notion MCP



Cursor docs

Source-backed narrative matters more than unsupported hype.